

Database Management System

B.Tech. II Year II Semester

UNIT - I

Database System Applications: A Historical Perspective, File Systems versus a DBMS, the Data Model, Levels of Abstraction in a DBMS, Data Independence, Structure of a DBMS

Introduction to Database Design: Database Design and ER Diagrams, Entities, Attributes, and Entity Sets, Relationships and Relationship Sets, Additional Features of the ER Model, Conceptual Design with the ER Model

UNIT - II

Introduction to the Relational Model: Integrity constraint over relations, enforcing integrity constraints, querying relational data, logical data base design, introduction to views, destroying/altering tables and views.

Relational Algebra, Tuple relational Calculus, Domain relational calculus.

UNIT - III

SQL: QUERIES, CONSTRAINTS, TRIGGERS: form of basic SQL query, UNION, INTERSECT, and EXCEPT, Nested Queries, aggregation operators, NULL values, complex integrity constraints in SQL, triggers and active data bases.

Schema Refinement: Problems caused by redundancy, decompositions, problems related to decomposition, reasoning about functional dependencies, FIRST, SECOND, THIRD normal forms, BCNF, lossless join decomposition, multi-valued dependencies, FOURTH normal form, FIFTH normal form.

UNIT - IV

Transaction Concept, Transaction State, Implementation of Atomicity and Durability, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for serializability, Lock Based Protocols, Timestamp Based Protocols, Validation- Based Protocols, Multiple Granularity, Recovery and Atomicity, Log-Based Recovery, Recovery with Concurrent Transactions.

UNIT - V

Data on External Storage, File Organization and Indexing, Cluster Indexes, Primary and Secondary Indexes, Index data Structures, Hash Based Indexing, Tree base Indexing, Comparison of File Organizations, Indexes and Performance Tuning, Intuitions for tree Indexes, Indexed Sequential Access Methods (ISAM), B+ Trees: A Dynamic Index Structure.

UNIT – I

Introduction to Database System Applications & Database Design

What is Data?

Data is a collection of a distinct small unit of information. It can be used in a variety of forms like text, numbers, media, bytes, etc. it can be stored in pieces of paper or electronic memory, etc. In computing, Data is information that can be translated into a form for efficient movement and processing.

What is Database? : Database is an organized collection of structured inter-related data, typically stored electronically in a computer system which helps in efficient retrieval, insertion and deletion of data from database and organizes the data in the form of tables, views, schemas, reports etc.

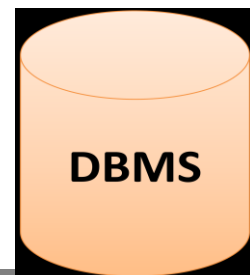
“A database is usually controlled by a Database Management System.”

For Example, university database organizes the data about **students, faculty, and admin staff** etc. which helps in **efficient retrieval, insertion and deletion of data** from it.

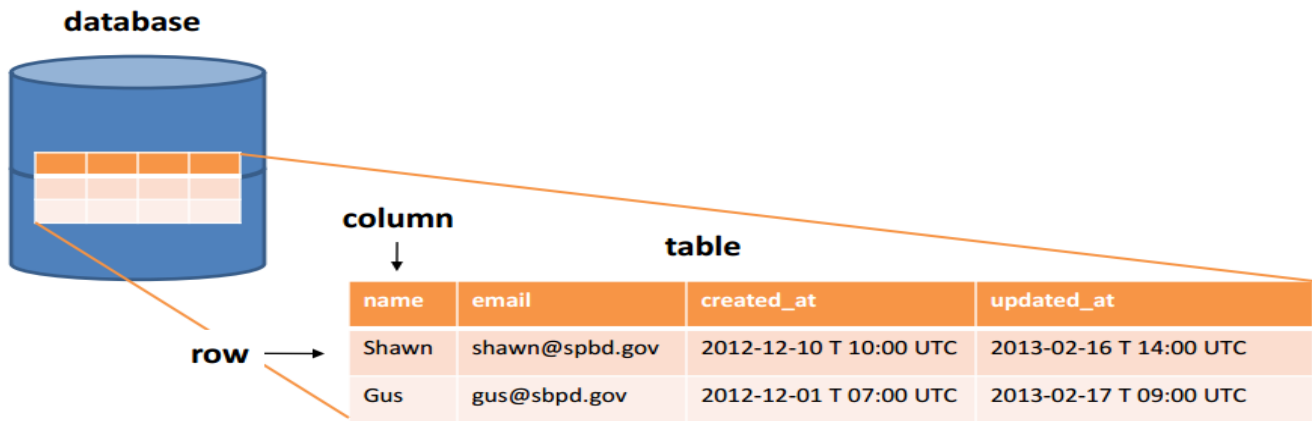
Database Management System: The software which is used to manage database is called Database Management System (DBMS). Data within the most common types of databases in operation today is *typically modeled in rows and columns* in a series of tables to make processing and data querying efficient. The data can then be easily accessed, managed, modified, updated, controlled, and organized.

Most databases use **Structured Query Language (SQL)** for writing and querying data.

Database Management System – DBMS is a collection of interrelated data and a set of programs to access and manipulate data. The primary goal of DBMS is convenient and effective store, retrieve and manipulate data.



A DBMS typically organizes the data in Rows and Column



LIST OF POPULAR DATABASES



A **Database Management System (DBMS)** is a software package designed to define, manipulate, retrieve and manage data in a database.

COMPONENTS OF DBMS



1. Hardware: All the physical devices that are being used in DBMS operations are called hardware.

2. Software: This is one of the most important components as it contains all the programs that will drive the DBMS functionality. The job of this software is to understand the Database access language and interpret actual database commands to execute them on the DB.

3. Data: The DBMS system collects, store, process and reads the data. This is yet another important component of the DBMS system. The DBMS contains the actual, operation or the metadata.

4. Procedures: Procedure is the general instructions and rules that help in using a DBMS.

5. Database Access Language: It is a query language that is used to write commands to perform CRUD operations like create, read, update and delete.

Applications of DBMS

APPLICATIONS OF DBMS



- 1. Banking System:** DBMS stores customers' usual and vital information, stores the transactions, user's activities, loans, accounts, etc.
- 2. Telecom:** It keeps a track of the calls that are made, network usage, details of the customer, bills, and balances, etc.
- 3. Airlines:** It stores flight information, customer details, flight timings, reservation and booking information, etc.
- 4. Online Shopping:** It stores product information, user's information, credit information, etc.
- 5. Educational and Corporate:** It stores students and employee information, client's details, training and course information, student/employee performance, etc.
- 6. Manufacturing and Industry:** It is used to keep a track of all units that are manufactured, products that go out and the ingredients that come in, etc.

MAIN FUNCTIONS OF A DBMS

Data Definition – Defines the Structure of Database.

Data Retrieval – Data extraction by using tools (SQL)

Data Manipulation – add/delete/modify the data in database

Access Control – Restrict unauthorized users to access data in database (Levels of Abstraction)

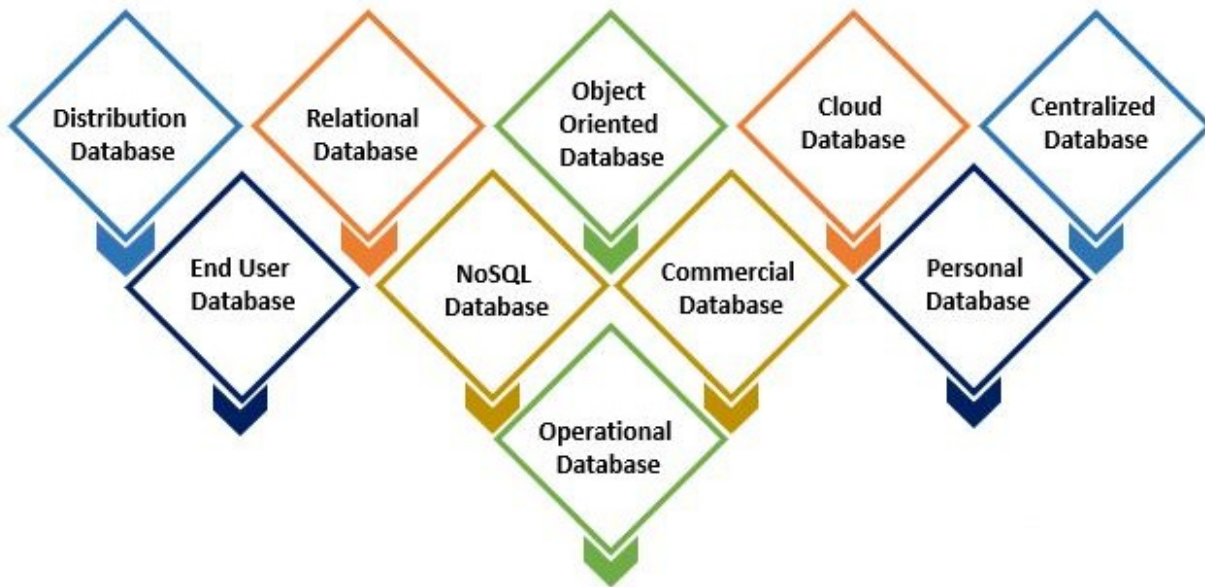
Data Sharing – A dbms allows multiple users to access data

Data Integrity – Maintains data integrity, Changes to data follow certain rules.

Security – In dbms, Security is used to grant /revoke privileges to users

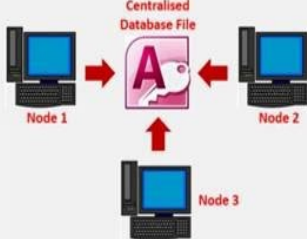
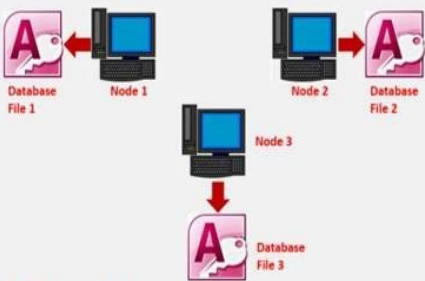
Recovery Crashes – Provides a mechanism for recovering the data in case of damage

TYPES OF DATABASE



1. **Centralized Database** – Access from multiple places (One Database + Multiple users)
2. **Distributed Database** – Multiple Database + Multiple Users
3. **Personal Database** – One Database + Single User
4. **End User Database** – Access by using application interfaces (For whom the application is developed)
5. **Commercial Database** – User have to pay for accessing data from Databases
6. **Relational Database** – Data is organized in rows and columns, represented in Table
7. **Cloud Database** – Data is stored in cloud , it is build for Virtual Environment
8. **Object Oriented Database** – Data is represented in classes and object (OOPS + RDBMS)
9. **Graph Database** – Collection of nodes and edges. Node represents Entity and Edge represents relationships between entities
10. **Operational Database** – Used to store day to day operational data / transactions data.
11. **NoSQL Database** - NoSQL databases ("not only SQL") are non tabular, and store data differently than relational tables.

Centralised vs. Distributed Databases

Centralised Databases	Distributed Databases
A single database located at 1 site on a network  Advantages: Since there is only 1 database file, it is easier to: • Get a complete view of Data • Manage, update and backup Data Disadvantages: • Bottle necking from multiple users accessing the same file – slowing down productivity	Consists of 2 or more files located at different sites on a network  Advantages: Having multiple database files means: • Users won't interfere with each other when accessing / manipulating Data • Speed since files are retrieved from nearest location • If one site fails, the system can still run Disadvantages: • Time for Synchronisation of the multiple databases • Data Replication for each different database file

DIFFERENT TYPES OF DATABASE USERS

The person who is using the Database is known as Database users he may be a scientist, engineer, employee, businessman, employee or layman

1. **Database Administrator (DBA)** – The person who is responsible for authorization Database (full control over the database). He has the privileges to add or remove the users.
2. **System Analysts** – The user's who fulfill the requirements of the end-users like naïve and parametric end users.
3. **Application Programmers** – They are the developers who built the software or applications to use that database properly.
4. **Naïve Users/Parametric Users** – These users are unsophisticated users who don't have any knowledge about databases. (Ex: clerk, admin staff)

5. **Sophisticated Users** – These users interact with the database but do not write programs. They are familiar with database but not with the coding section.
6. **Database designer** – Database designers are responsible for the structure of data where we store the data and share the data.
7. **Casual Users / Temporary Users** – These type of users communicate with databases for a period of time.

File Processing System VS DBMS

File Processing System - In Computer Science, File Processing System (FPS) is a way of storing, retrieving and manipulating data which is present in various files. Files are used to store various documents. In file processing system, if one needs to insert, delete, modify, store or update data, one must know the entire hierarchy of the files.

Disadvantages of File Processing System:

1. **Data Redundancy and Inconsistency** - Multiple file formats, duplication of information in different files refers to Data Redundancy. Data inconsistency is a condition that occurs when the same data exists in different formats in multiple tables. **Inconsistency** is generally compounded by **data** redundancy.
2. **Difficulty in Accessing Data** - Accessing data is not convenient and efficient in file processing system. Need to write a new program to carry out each new task.
3. **Data Isolation** – All related data is not available in single file. Data is available in multiple files with different formats. Writing new programs to retrieve appropriate data is difficult.
4. **Integrity Problems** - Data integrity means that the data contained in the database is both correct and consistent. For this purpose, the data stored in database must satisfy existing constraints.

Integrity constraints (e.g. account balance > 0) become part of program code. Hard to add new constraints or change existing ones.

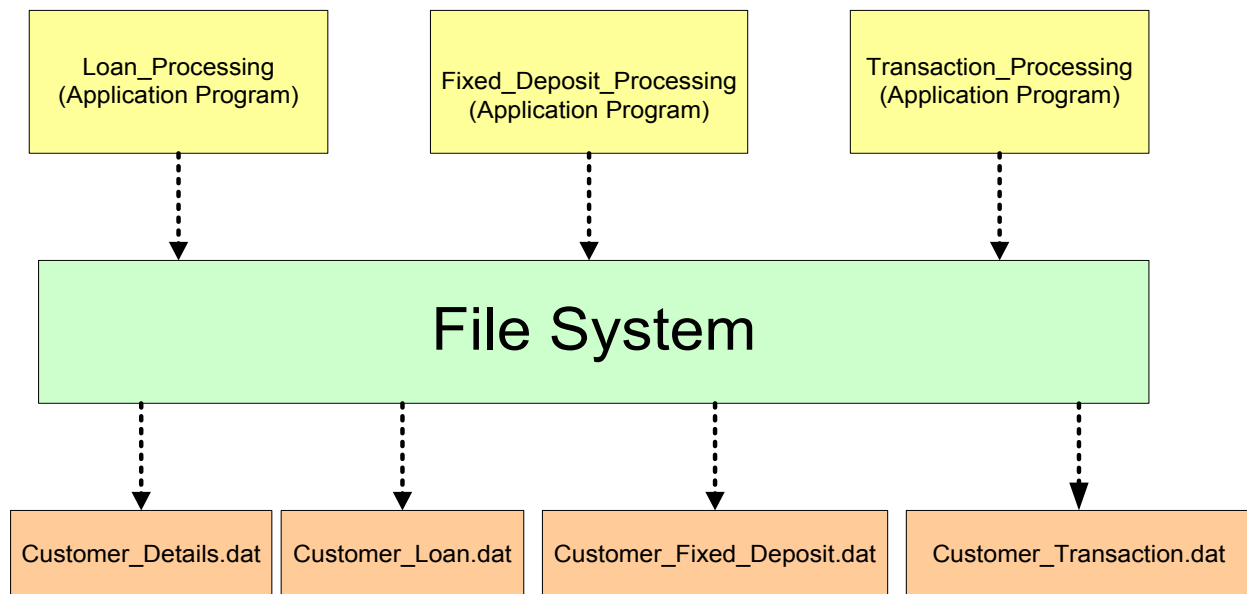
5. **Atomicity Problems**: Any operation on database must be atomic. It means that, it must happen either completely or not at all. Failures may leave database in an inconsistent state with

partial updates carried out. (Eg: Transfer of funds from one account to another should either complete or not happen at all)

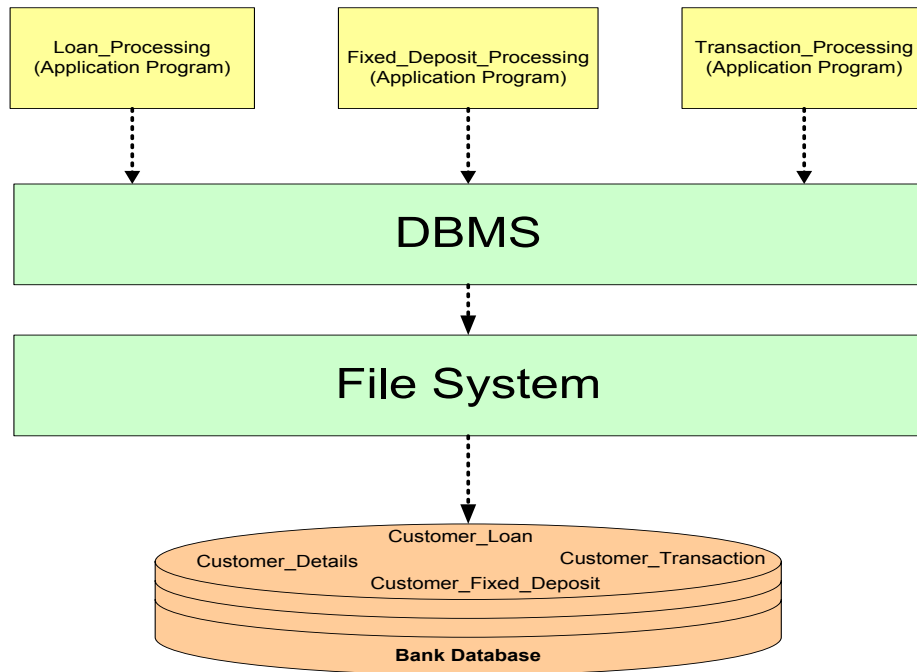
6. **Concurrent access anomalies:** Multiple users are allowed to access data simultaneously for the sake of better performance and faster response. Uncontrolled concurrent accesses can lead to inconsistencies. (E.g. two people reading a balance and updating it at the same time).
7. **Security Problems** – Data is easily accessible by all. Database should be accessible to users in limited way. Each user should be allowed to access data concerning his requirements only
8. **Limited Data Sharing** - Data are scattered in various files and also in different files may have different formats and these files may be stored in different folders may be of different departments. So, due to this data isolation, it is difficult to share data among different applications.

Note: Database Management System (DBMS) is used to eliminate the disadvantages of the FPS.

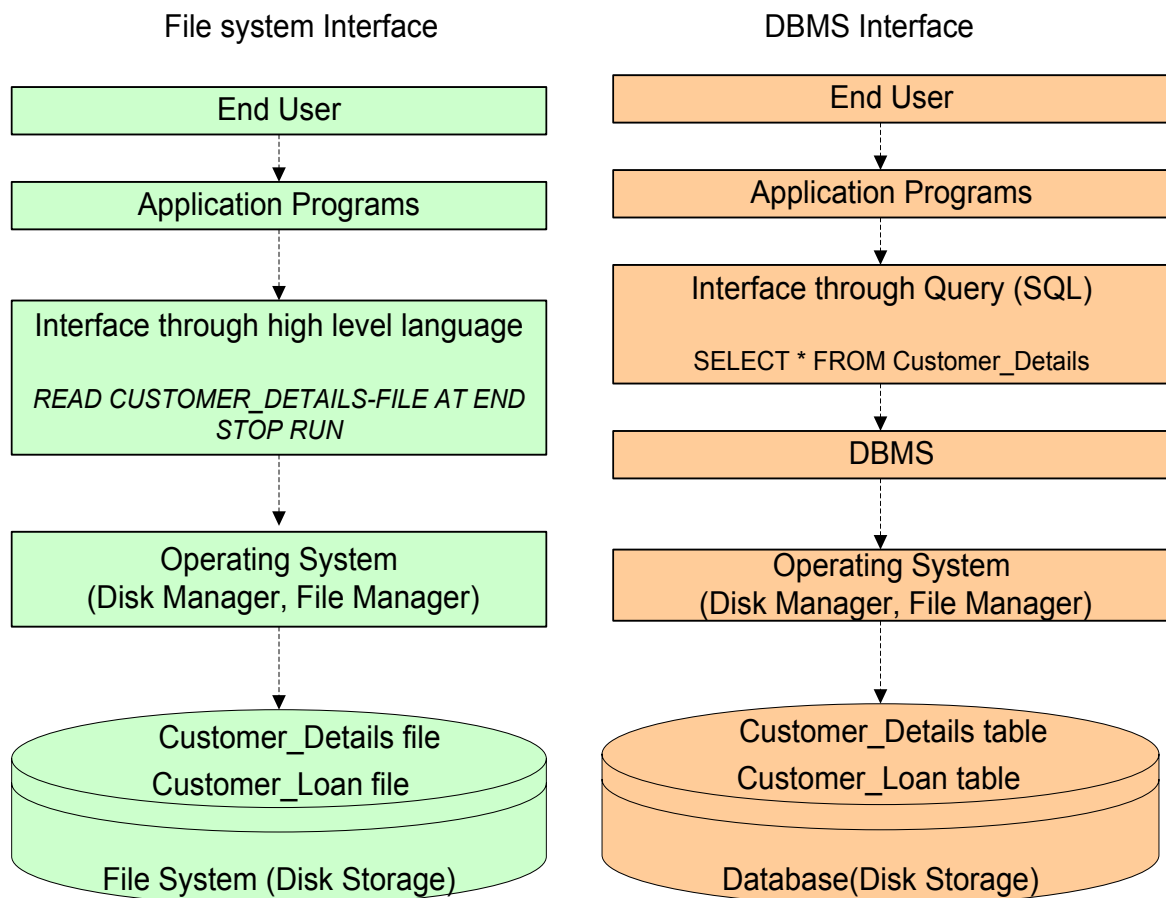
Traditional Method of Data Storage



Position of DBMS



Difference between File and DBMS Operations



Advantages of DBMS

1. **Data Redundancy** - **Data** Redundancy occurs when the same data are stored unnecessarily at different places. Data Redundancy is reduced or eliminated in DBMS because all data are stored at a centralized location rather than being created by individual users and for each application
2. **Data Inconsistency** - In traditional file system storage, the changes made by one user in one application doesn't update the changes in the other application, given both have the same set of details. While this is not the case with DBMS systems as there is a single repository of data that is defined once and is accessed by many users, and data are consistent.
3. **Data Sharing** - Data Sharing is the primary feature of Database management systems. DBMS system allows users and applications to share Data with multiple applications and users. Data are stored in one or more servers in the network and there is some software locking mechanism that prevents the same set of data from being changed by two people at the same time. While the file system doesn't have this capability.
4. **Data Searching** - Searching and retrieving data is very easy in DBMS systems. The need to write separate programs for each of the search is eliminated as in the case with a traditional file-based approach. In DBMS, we can write small queries to search for multiple data at a time from Database servers.
5. **Data Security** - DBMS systems provide a strong framework to protect data privacy and security. DBMS ensures that only authorized users have access to data and there is a mechanism to define access privileges.
6. **Data Concurrency** - In DBMS, Data are stored in one or more servers in the network and that there is some software locking mechanism that prevents the same set of data from being changed by two people at the same time.
7. **Data Integration** - Data integration is a process of combining the data residing at different locations and presents the user with a unified view of data. DBMS systems allow Data Integration with much feasibility.
8. **Data Access** - While in a traditional file-based approach, it might take hours to look for very specific information that might be needed in the context of some business emergency, while DBMS reduces this time to a few seconds. We can write small queries which will search the

Database for you and it will retrieve the information in the fastest way possible due to its inbuilt searching operations.

9. **Data Backup and Recovery** - DBMS provides a strong framework for Data backup, users are not required to back up their data periodically and manually, it is automatically taken care of by DBMS. Moreover, in case of a server crash, DBMS restores the Database to its previous condition.
10. **Data Atomicity** - An atomic transaction is one in which all of the database actions occur or none of them does. It is the duty of DBMS to store a complete transaction in the database. If any transaction is partially completed, then it rolls back them. For e.g. if we make an online purchase, money is deducted from our account while if the purchase is somehow failed, then no money is deducted or if it gets deducted, it gets returned within few days.

Disadvantages of DBMS

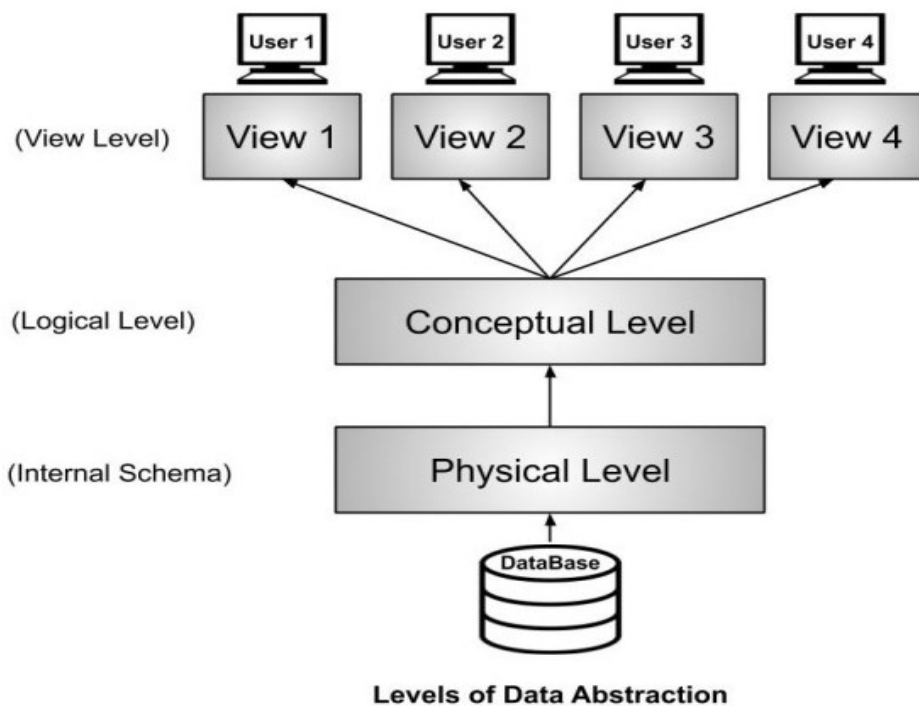
- Cost of hardware and software
- Technical staff requirement
- Management Complexity
- Frequent upgrade/Management cycles

Levels of Abstraction in DBMS / View of Data in DBMS

Abstraction is one of the main features of database systems. Hiding irrelevant details from user and providing abstract view of data to users, helps in easy and efficient **user-database** interaction. Database systems are made-up of complex data structures. To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called **Data Abstraction**.

There are mainly three levels of data abstraction and we divide it into three levels in order to achieve **Data Independence**. **Data Independence** means users and data should not directly interact with each other. The three levels of data abstraction are

1. **View Level**
2. **Conceptual Level**
3. **Physical Level**



1. **View Level or External Schema:** Highest level of data abstraction. This level describes the user interaction with database system. At **view level**, user just interact with system with the help of GUI and enter the details at the screen, they are not aware of how the data is stored and what data is stored; such details are hidden from them.

Example: If we have a login-id and password in a university system, then as student, we can view our marks, attendance, fee structure, etc. But the faculty of the university will have a different view. He will have options like salary, edit marks of a student, enter attendance of the students, etc. So, both the student and the faculty have a different view

2. **Conceptual Level or Logical Level:** This is the middle level of 3-level data abstraction architecture. It describes what data is stored in database and how the data is actually stored and structured. We have different data models by which we can store the data. At the logical level these records can be described as fields and attributes along with their data types, their relationship among each other can be logically implemented.

Example: Let us take an example where we use the relational model for storing the data. We have to store the data of a student, the columns in the student table will be student_name, age, mail_id, roll_no etc. We have to define all these at this level while we are creating the database

3. **Physical level:** This is the lowest level of data abstraction. It describes how data is actually stored in database.

Example: Let's say we are storing customer information in a customer table. At **physical level** these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.

Customer_Loan Cust_ID : 101 Loan_No : 1011 Amount_in_Dollars : 8755.00	External
CREATE TABLE Customer_Loan (Cust_ID NUMBER(4) Loan_No NUMBER(4) Amount_in_Dollars NUMBER(7,2))	Conceptual
Cust_ID TYPE = BYTE (4), OFFSET = 0 Loan_No TYPE = BYTE (4), OFFSET = 4 Amount_in_Dollars TYPE = BYTE (7), OFFSET = 8	Internal

DBMS Instance

Definition of Instance: The data stored in database at a particular moment of time is called instance of database. Database schema defines the variable declarations in tables that belong to a particular database; the value of these variables at a moment of time is called the instance of that database.

DBMS Schema

Definition of Schema: Design of a database is called the schema. Schema is of three types: Physical Schema, Logical Schema and View Schema

Consider an Example of a University Database. At the different levels this is how the implementation will look like:

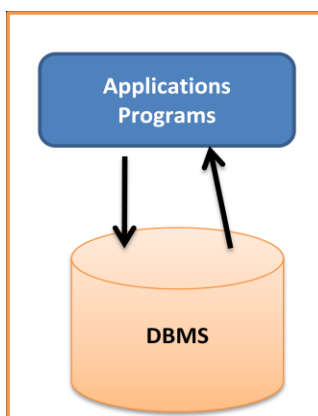
Type of Schema	Implementation
External Schema	View 1: Course info(cid:int,cname:string) View 2: <u>studentinfo(id:int, name:string)</u>
Conceptual Schema	Students(id: <u>int</u> , name: string, login: string, age: integer) Courses(id: <u>int</u> , <u>cname.string</u> , <u>credits:integer</u>) Enrolled(id: <u>int</u> , <u>grade:string</u>)
Physical Schema	•Relations stored as unordered files. •Index on the first column of Students.

DBMS Architecture - The architecture of DBMS depends on the computer system on which it runs.

Types of DBMS Architecture

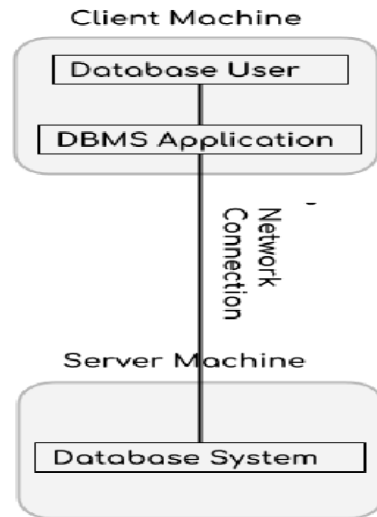
There are three types of DBMS architecture:

1. **Single Tier Architecture** - In this type of architecture, the database is readily available on the client machine, any request made by client doesn't require a network connection to perform the action on the database

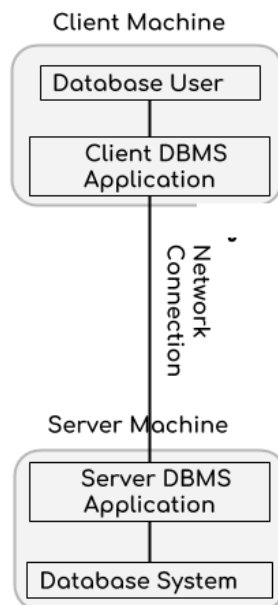


2. Two Tier Architecture

A Two Tier Architecture in DBMS is a Database architecture where the presentation layer runs on a client (PC, Mobile, Tablet, etc.), and data is stored on a server called the second tier. Two tier architecture provides added security to the DBMS as it is not exposed to the end-user directly



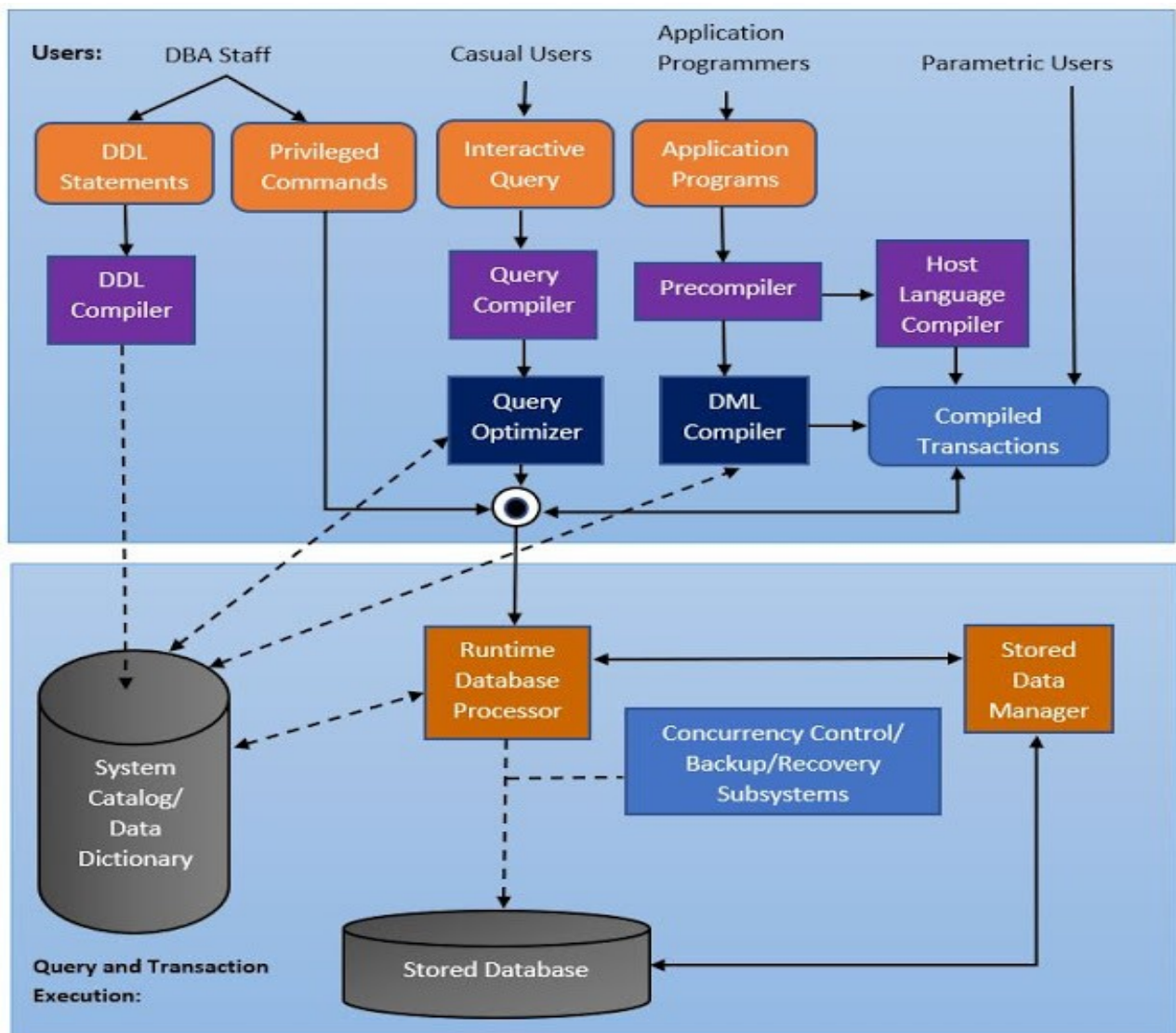
3. **Three Tier Architecture** – It is a type of software architecture which is composed of three “tiers” or “layers” of logical computing. They are often used in applications as a specific type of client-server system. 3-tier architectures provide many benefits for production and development environments by modularizing the user interface, business logic, and data storage layers



Database System Architecture/ Structure of DBMS

Database Management System is software that allows access to data stored in a database and provides an easy and effective method of –

1. Defining the information.
2. Storing the information.
3. Manipulating the information.
4. Protecting the information from system crashes or data theft.
5. Differentiating access permissions for different users.



Architecture is divided into two major blocks.

First block represents the different types of end users and their respective interfaces for interacting with the database system.

Second block shows the internal components responsible for the storage management of data and processing of transactions.

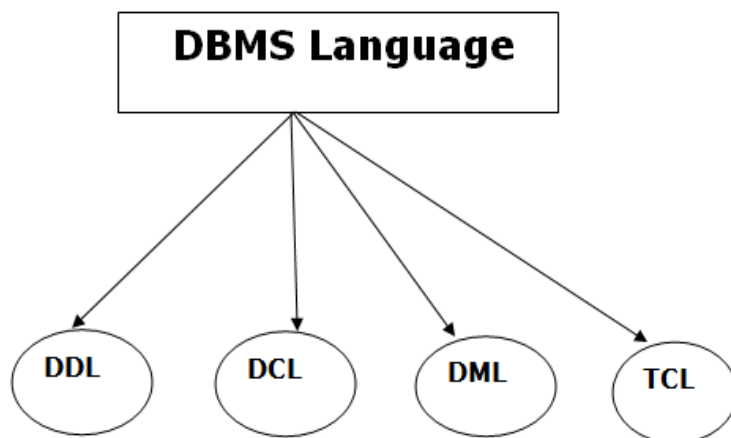
1. **DBA Staff:** Database administrative staff, defines database structure.
2. **DDL Statements and Privileged Commands:** DBA staff use DDL (Data Definition Language) statements and Privileged Commands (Access control statements allocated as per assigned roles) to define database structure and make changes in definitions as per the requirements on timely basis.
3. **DDL Compiler:** It compiles DDL statements and record database definitions in system catalog or data dictionary.
4. **Data Dictionary/Storage Catalog:** Data dictionary store meta data (data about data). Meta data includes the structure of the database (information about schema) and the details like names, data types, sizes of data elements. It also stores the relations in between the data items along with different type of constraints enforced.
5. **Casual Users:** Write queries to interact with the database system.
6. **Interactive Query Interface:** User friendly interface for casual users to write queries.
7. **Query Compiler:** Query written by casual user compiled by query compiler. Query compiler parses the query and checks the query syntax for correctness. It translates query into some internal form.
8. **Query Optimizer:** It takes the internal form of a query from query compiler and rearrange the operations, remove redundancies, apply efficient algorithms and indexes for fast and optimized access of stored data. It creates fast and efficient query plans for improving the performance.
9. **Application Programmers:** Writes application programs using some high level programming language like C/C++, Pascal, Java etc.
10. **Application Programs:** A program designed to perform any specific database function as per any user specific requirement or for any other application program. These programs are submitted to Precompiler.

- 11. Precompiler:** It takes out the DML statements written in application program and pass them to the DML compiler. It sends remaining program statements to host language compiler.
- 12. DML Compiler:** It translates the DML statements into object code for database access.
- 13. Host Language Compiler:** High level language compiler in which application programs are written.
- 14. Compiled Transactions:** Compiled transactions include the output of linked DML object codes and high level language statements from application program. These transactions are known as canned transactions. These canned transactions then calls runtime database processor.
- 15. Parametric Users:** Data entry operators, Interact database by passing parameters regularly to canned transactions for accessing data.
- 16. Runtime Database Processor:** It takes all the query plans, compiled transactions with parameters and privileged commands and executes them with the help of stored data manager and data dictionary. It also support internal buffer management (if exist).
- 17. Stored Data Manager:** Manages and control the access of information that is stored in disk or in storage catalog.
- 18. Concurrency Control and Backup/Recovery Subsystem:** This component is integrated with runtime database processor to perform transaction management. It is also known as **transaction manager**.

Database Language - Database languages are used to *read, update and store data in a database*.

There are several such languages that can be used for this purpose; one of them is SQL (Structured Query Language).

Types of Database Language



Data Definition Language (DDL)

DDL is used for specifying the database schema. It is used for creating tables, schema, indexes, constraints etc. in database. The following are the operations that we can perform on database using DDL:

To create the database instance – **CREATE**

To alter the structure of database – **ALTER**

To drop database instances – **DROP**

To delete tables in a database instance – **TRUNCATE**

To rename database instances – **RENAME**

To drop objects from database such as tables – **DROP**

To Comment on the data dictionary – **Comment**

Data Manipulation Language (DML)

DML is used for accessing and manipulating data in a database. The following operation on database comes under DML:

To read records from table(s) – **SELECT**

To insert record(s) into the table(s) – **INSERT**

Update the data in table(s) – **UPDATE**

Delete all the records from the table – **DELETE**

Data Control language (DCL)

DCL is used for granting and revoking user access on a database –

To grant access to user – **GRANT**

To revoke access from user – **REVOKE**

Transaction Control Language (TCL)

The changes in the database that we made using DML commands are either performed or roll backed using TCL.

To persist the changes made by DML commands in database – **COMMIT**

To rollback the changes made to the database – **ROLLBACK**

Data Models in DBMS

Data Model is a logical structure of Database. It describes the design of database to reflect entities, attributes, relationship among data, constraints etc.

Types of Data Models

Object based logical Models – Describe data at the conceptual and view levels.

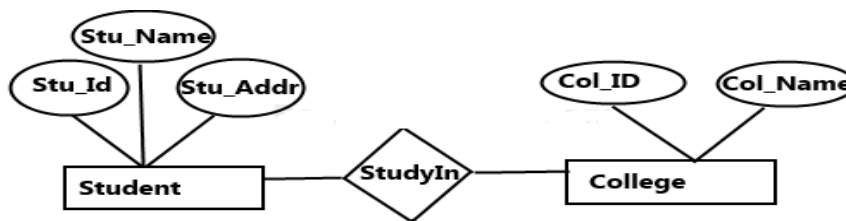
1. E-R Model
2. Object oriented Model

Record based logical Models – Like Object based model, they also describe data at the conceptual and view levels. These models specify logical structure of database with records, fields and attributes.

1. Relational Model
2. Hierarchical Model
3. Network Model

Object based logical Models – Describe data at the conceptual and view levels.

1. **E-R Model** - An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram). The main components of E-R model are: entity set and relationship set.

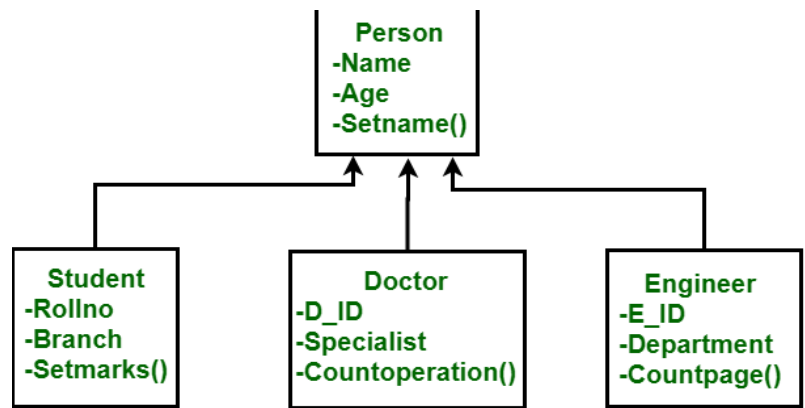


Sample E-R Diagram

2. **Object oriented Model** - In Object Oriented Data Model, data and their relationships are contained in a single structure which is referred as object in this data model. In this, real world problems are represented as objects with different attributes. All objects have multiple relationships between them. Basically, it is combination of Object Oriented programming and Relational Database Model

Components of Object Oriented Data Model:

1. Objects
2. Attribute
3. Methods
4. Class

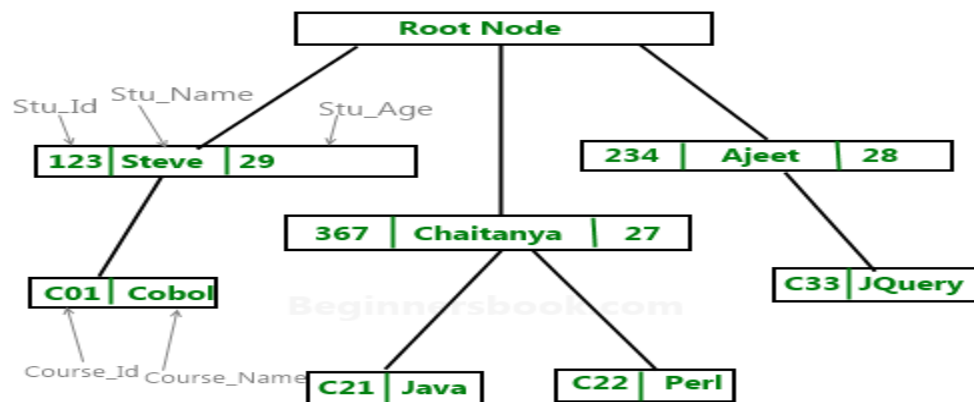


Record based logical Models – Like Object based model, they also describe data at the conceptual and view levels. These models specify logical structure of database with records, fields and attributes.

1. **Relational Model** - In relational model, the data and relationships are represented by collection of inter-related tables. Each table is a group of column and rows, where column represents attribute of an entity and rows represents records.

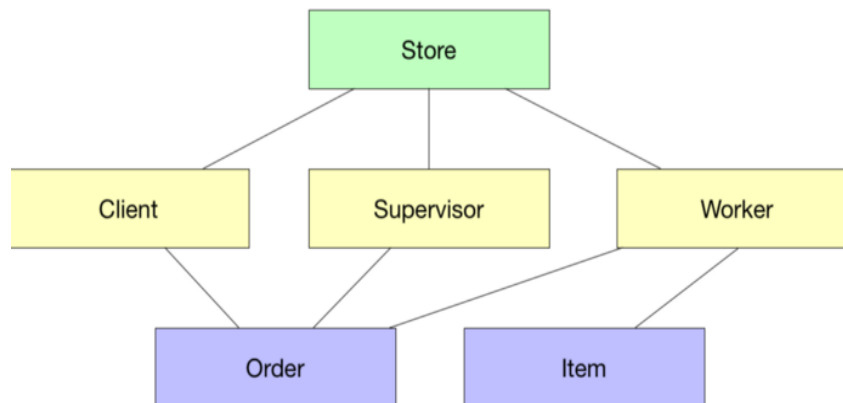
<u>Stu_Id</u>	Stu_Name	Stu_Age
111	Ashish	23
123	Saurav	22
169	Lester	24
234	Lou	26

2. **Hierarchical Model** - In **hierarchical model**, data is organized into a tree like structure with each record is having one parent record and many children. The main drawback of this model is that, it can have only one to many relationships between nodes



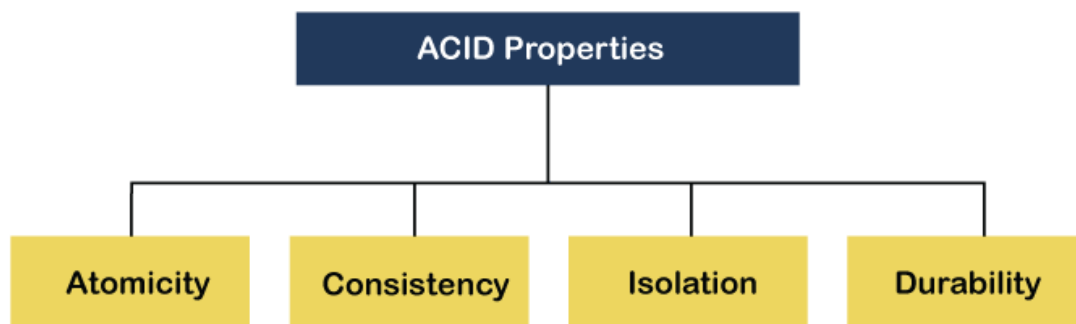
3. **Network Model** - Network Model is same as hierarchical model except that it has graph-like structure rather than a tree-based structure. Unlike hierarchical model, this model allows each record to have more than one parent record.

The Network Database Model

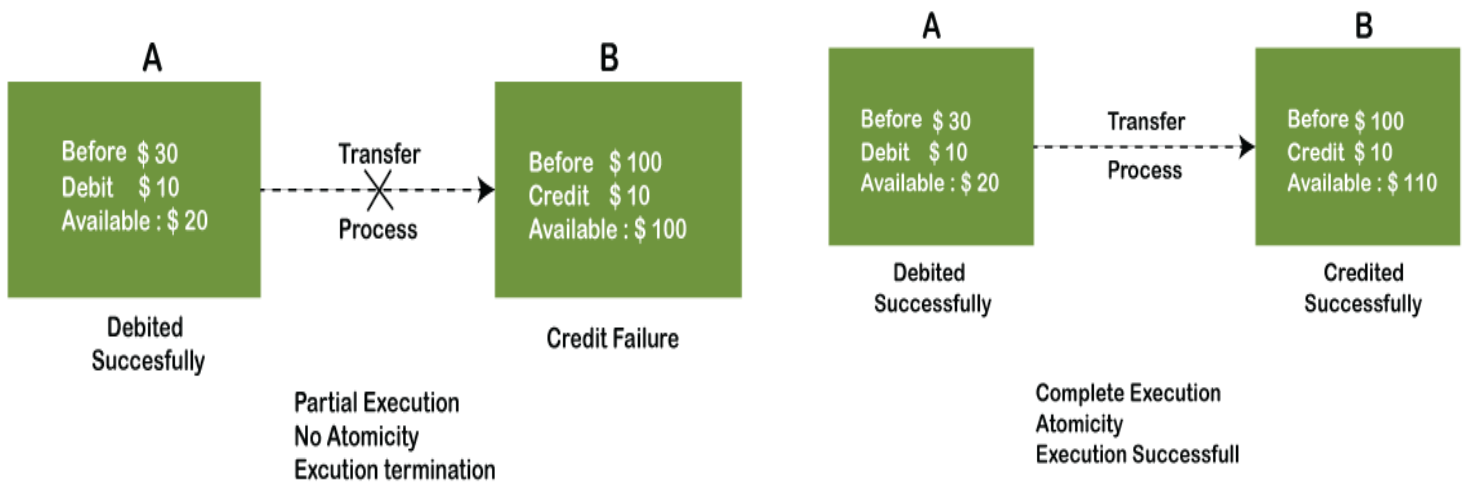


ACID Properties in DBMS

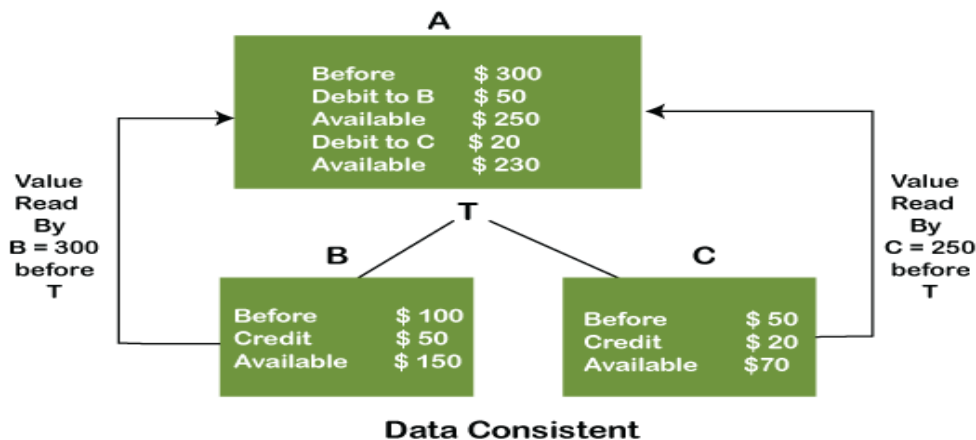
DBMS is the management of data that should remain integrated when any changes are done in it. It is because if the integrity of the data is affected, whole data will get disturbed and corrupted. Therefore, to maintain the integrity of the data, there are four properties described in the database management system, which are known as the **ACID** properties.



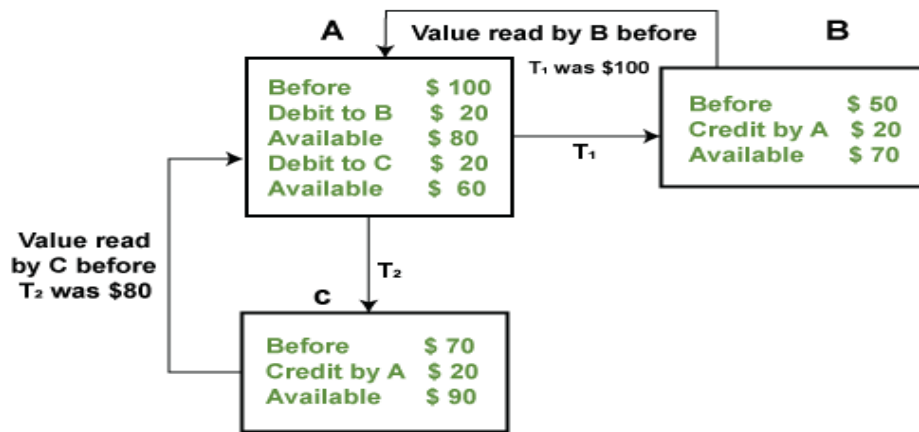
1) Atomicity: The term atomicity defines that the data remains atomic. It means if any operation is performed on the data, either it should be performed or executed completely or should not be executed at all. It further means that the operation should not break in between or execute partially. In the case of executing operations on the transaction, the operation should be completely executed and not partially.



2) Consistency: The word **consistency** means that the value should remain preserved always. In DBMS, the integrity of the data should be maintained, which means if a change in the database is made, it should remain preserved always. In the case of transactions, the integrity of the data is very essential so that the database remains consistent before and after the transaction. The data should always be correct.



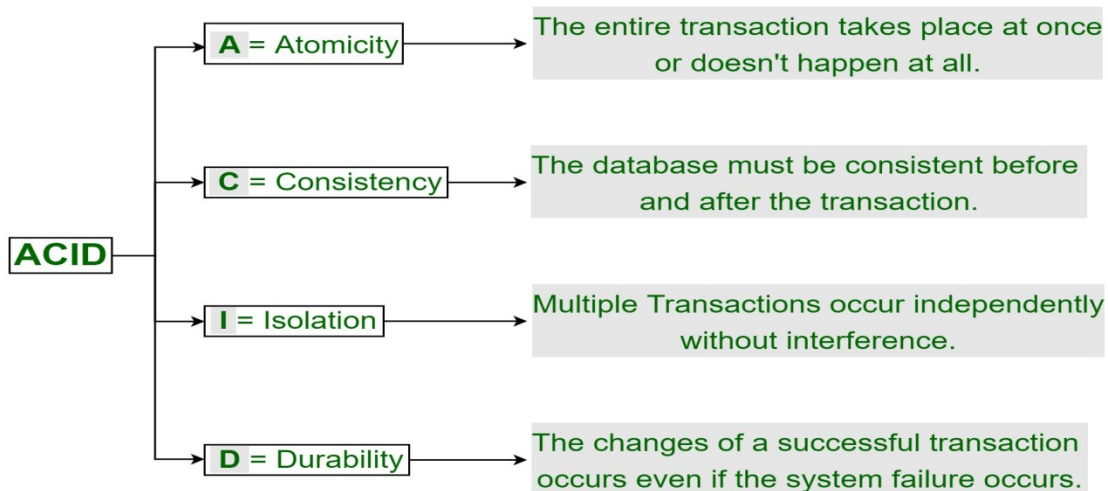
3) Isolation: The term 'isolation' means separation. In DBMS, Isolation is the property of a database where no data should affect the other one and may occur concurrently. In short, the operation on one database should begin when the operation on the first database gets complete. It means if two operations are being performed on two different databases, they may not affect the value of one another. In the case of transactions, when two or more transactions occur simultaneously, the consistency should remain maintained. Any changes that occur in any particular transaction will not be seen by other transactions until the change is not committed in the memory.



Isolation - Independent execution of T₁ & T₂ by A

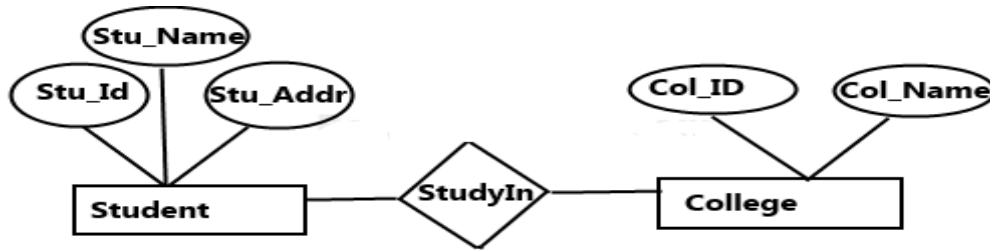
4) Durability: Durability ensures the permanency of something. In DBMS, the term durability ensures that the data after the successful execution of the operation becomes permanent in the database. The durability of the data should be so perfect that even if the system fails or leads to a crash, the database still survives. However, if gets lost, it becomes the responsibility of the recovery manager for ensuring the durability of the database. For committing the values, the COMMIT command must be used every time we make changes. Therefore, the ACID property of DBMS plays a vital role in maintaining the consistency and availability of data in the database.

ACID Properties in DBMS



Introduction to Database Design with E-R Model

E-R Model - An Entity-relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram). The main components of E-R model are: entity set and relationship set.



Sample E-R Diagram

Rectangle: Represents Entity sets

Ellipses: Attributes

Diamonds: Relationship Set

Lines: They link attributes to Entity Sets and Entity sets to Relationship Set

Double Ellipses: Multivalued Attributes

Dashed Ellipses: Derived Attributes

Double Rectangles: Weak Entity Sets

Double Lines: Total participation of an entity in a relationship set

Primary key: attributes are underlined



Entity or Strong Entity



Weak Entity



Attribute



Multivalued Attribute

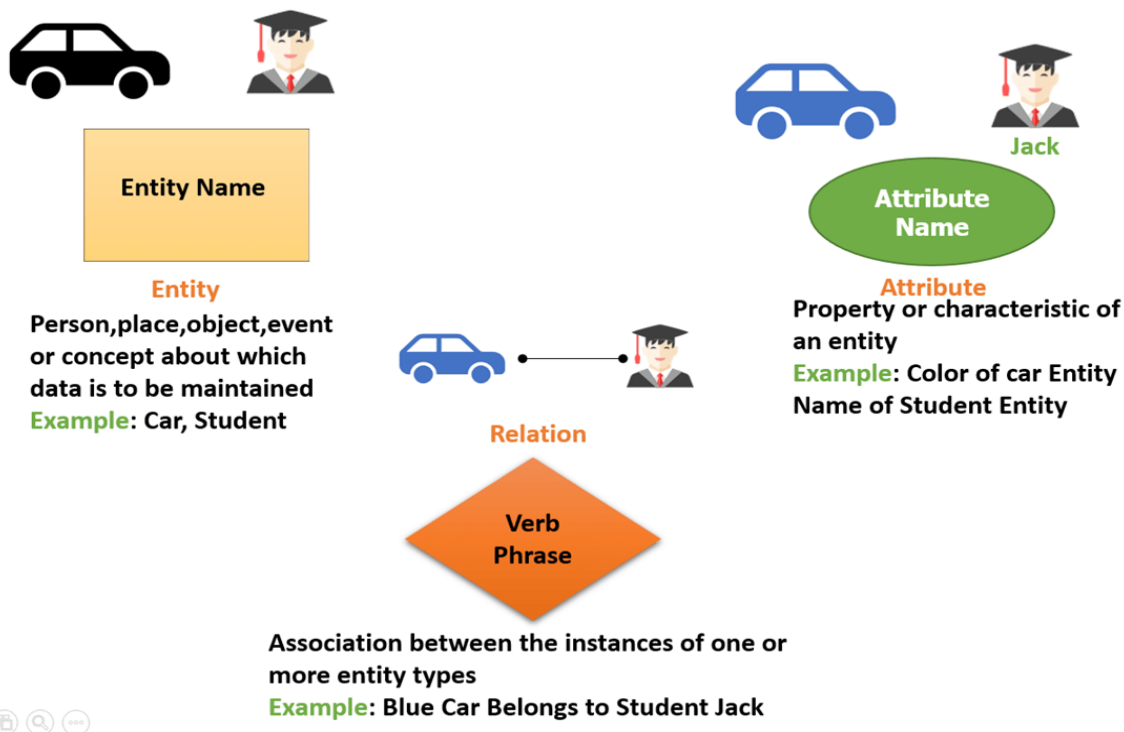


Relationship

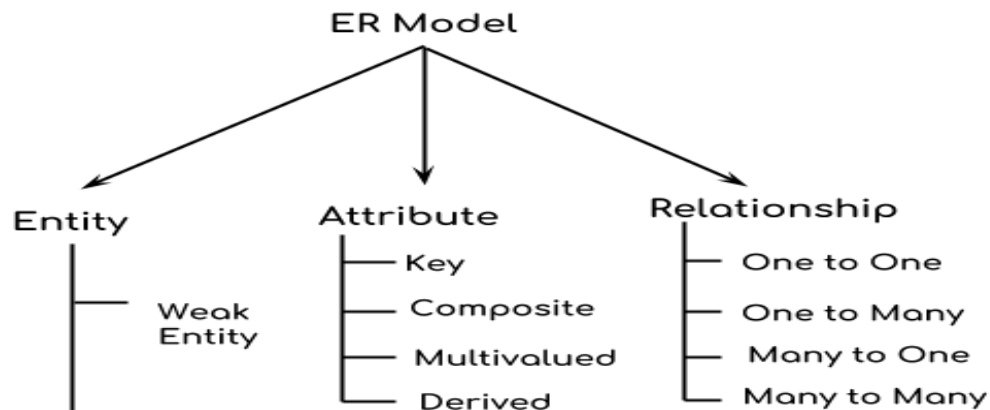


Weak Relationship





Components of an ER Diagram



As shown in the above diagram, an ER diagram has three main components:

1. **Entity**
2. **Attribute**
3. **Relationship**

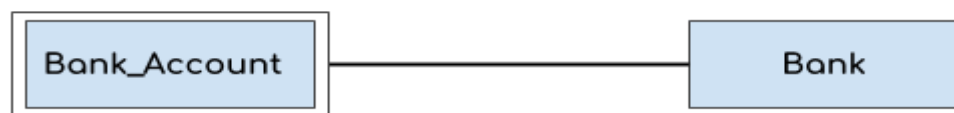
1. Entity -An entity is an object or component of data. An entity is represented as rectangle in an ER diagram. **For example:** In the following ER diagram we have two entities Student and College and these two entities have many to one relationship as many students study in a single college.



Weak Entity:

An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle.

For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



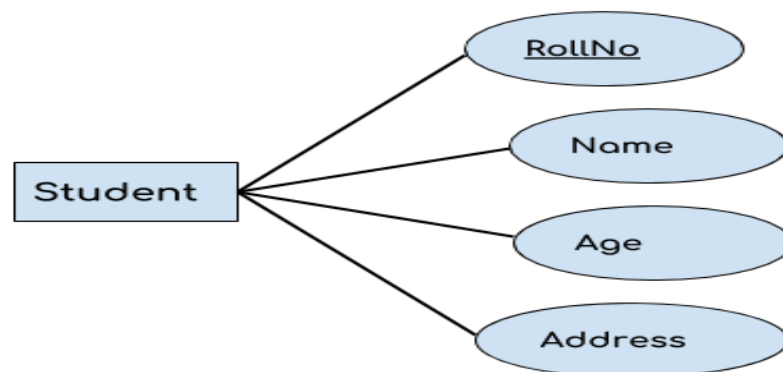
2. Attribute

An attribute describes the property of an entity. An attribute is represented as Ellipse in an ER diagram. There are four types of attributes:

1. Key attribute
2. Composite attribute
3. Multivalued attribute
4. Derived attribute

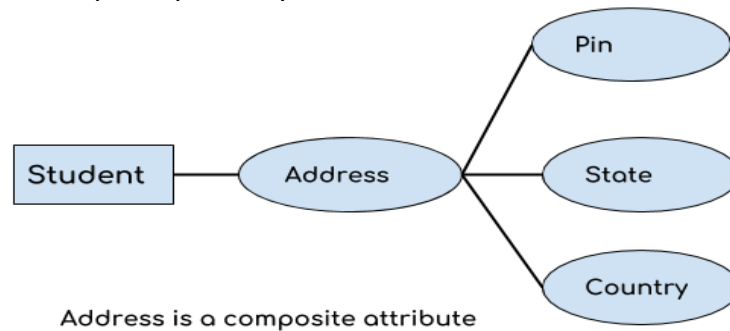
1. Key Attribute:

A key attribute can uniquely identify an entity from an entity set. Key attribute is represented by Ellipse same as other attributes however the text of key attribute is underlined.



2. Composite attribute:

An attribute that is a combination of other attributes is known as composite attribute. For example, In student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.



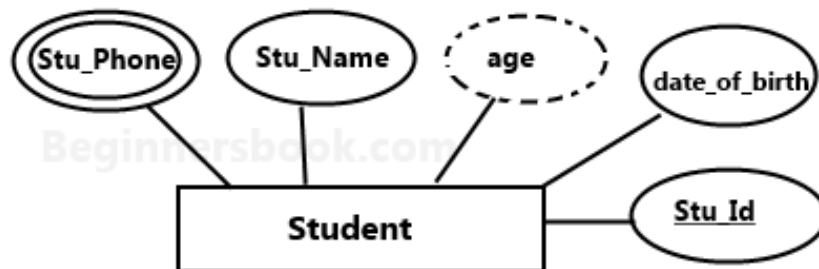
3. Multivalued attribute:

An attribute that can hold multiple values is known as Multivalued attribute. It is represented with **double Ellipses** in an ER Diagram. For example – A person can have more than one phone numbers so the phone number attribute is Multivalued.

4. Derived attribute:

A derived attribute is one whose value is dynamic and derived from another attribute. It is represented by **dashed Ellipse** in an ER Diagram. For example – Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).

E-R diagram with multivalued and derived attributes:



3. Relationship

A relationship is represented by diamond shape in ER diagram, it shows the relationship among entities. There are four types of relationships:

1. One-To-One

2. One-To-Many

3. Many-To-One

4. Many-To-Many

1. One-To-One Relationship

When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport and a passport is given to one person.

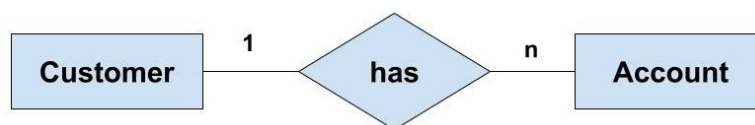


2. One-To-Many Relationship

When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship. For example – a customer can place many orders but a order cannot be placed by many customers.



For example, If there are two entity type 'Customer' and 'Account' then each 'Customer' can have more than one 'Account' but each 'Account' is held by only one 'Customer'. In this example, we can say that each Customer is associated with many Accounts. So, it is a **one-to-many relationship**. But, if we see it the other way i.e many Account is associated with one Customer then we can say that it is a **many-to-one relationship**.



3. Many-To-One Relationship

When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship. For example – many students can study in a single college but a student cannot study in many colleges at the same time.



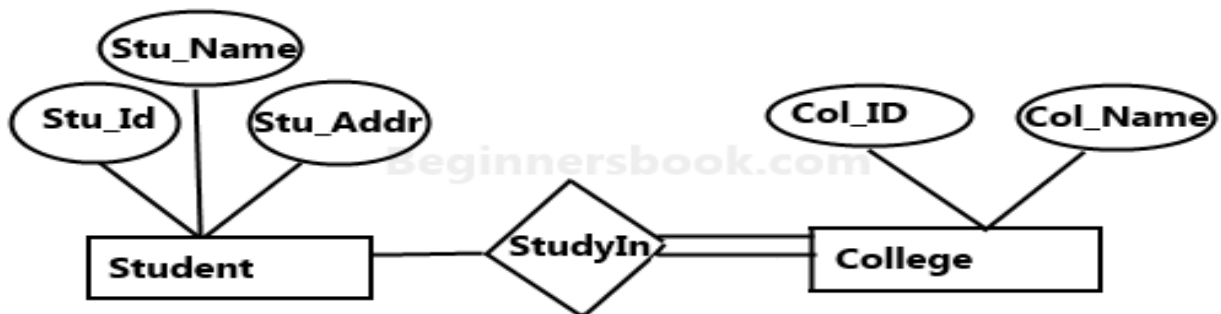
4. Many-To-Many Relationship

When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship. For example, a can be assigned to many projects and a project can be assigned to many students.



Total Participation of an Entity set

A Total participation of an entity set represents that each entity in entity set must have at least one relationship in a relationship set. For example: In the below diagram each college must have at-least one associated Student.



E-R Digram with total participation of College entity set in StudyIn relationship Set - This indicates that each college must have atleast one associated Student.

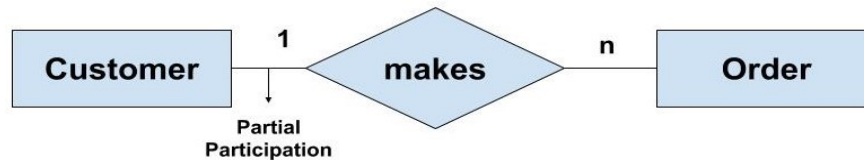
Participation Constraints

The relationship can be between two strong entity or a strong entity and a weak entity. Depending upon the type of entity participating in the relationship, the participation can be partial or total. There are two types of participation constraints:

- Partial Participation
- Total Participation

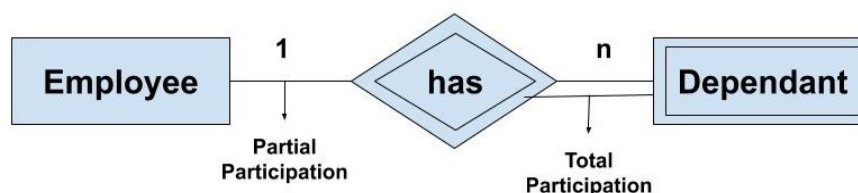
Partial Participation: Partial Participation exists when all the entity of an entity type is not associated with one or the other entity of another entity type. This is represented by joining the relationship with the entity type with the help of one line.

Example: We have two entity type 'Customer' and 'Order'. Then there can be 'Customer' who have not done any order. So, here there is partial participation of the entity in the relationship.



Total Participation: Total Participation exists when all the entity of an entity type is associated with one or the other entity of another entity type. This is represented by joining the relationship with the entity type with the help of a double parallel line. Such a relationship usually exist between a strong entity and a weak entity.

Example: We have two entity type 'Employee' and 'Dependant'. Then all the 'Dependant' entity are related to one or the other 'Employee' entity. This is called total participation of the entity in the relationship. But, it may be possible that some 'Employee' is not related to any of the 'Dependant' entity. So, 'Employee' is showing partial participation whereas the 'Dependant' is showing total participation in the relationship.

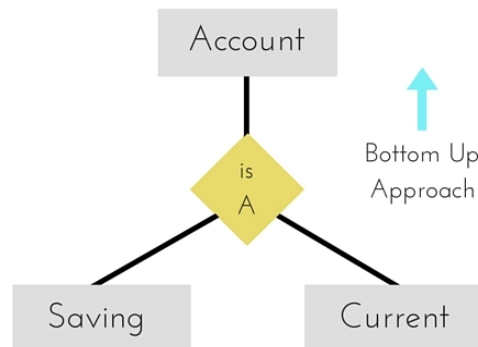


The Enhanced ER Model

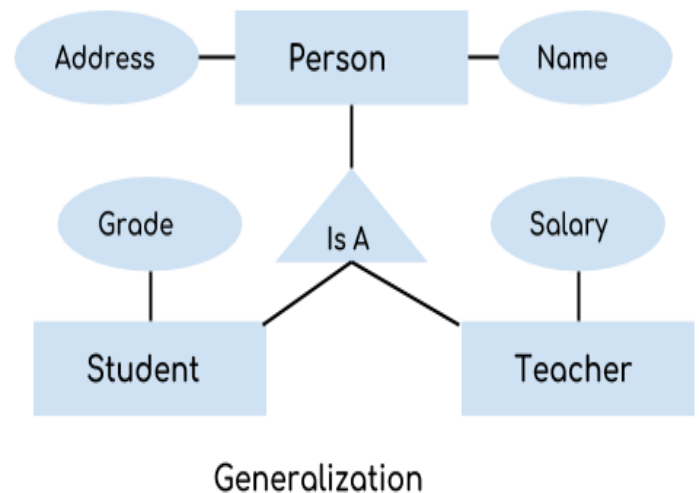
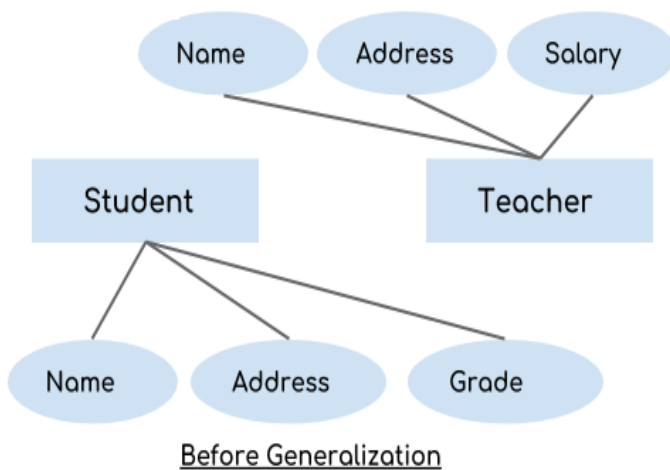
Hence, as part of the **Enhanced ER Model**, along with other improvements, three new concepts were added to the existing ER Model, they were:

1. Generalization
2. Specialization
3. Aggregation

1. Generalization - is a bottom-up approach in which two lower level entities combine to form a higher level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity. For example, **saving** and **Current** account types entities can be generalized and an entity with name **Account** can be created, which covers both.

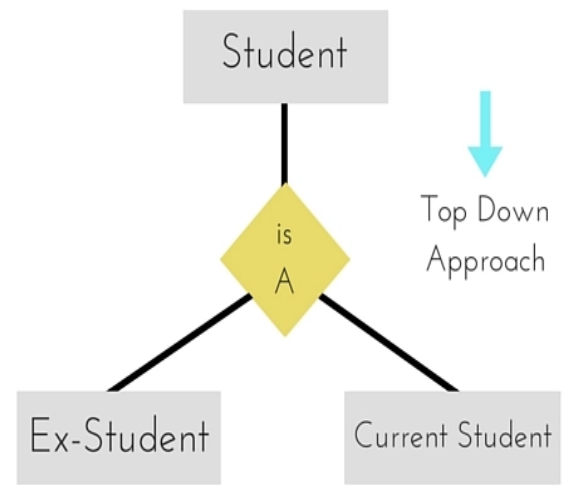
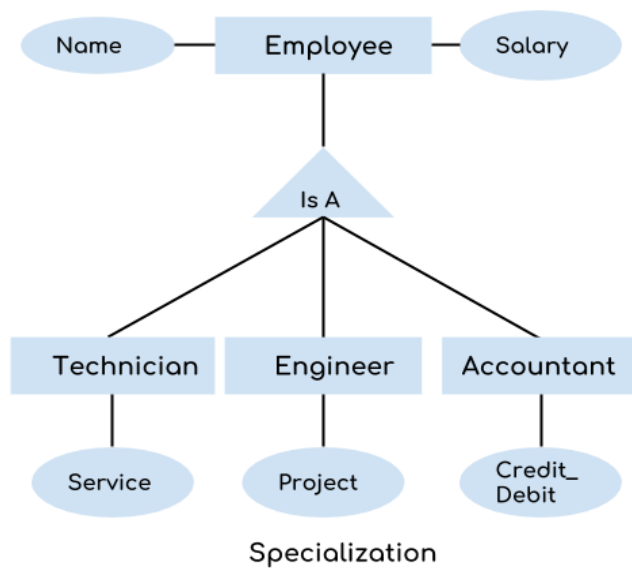


Generalization is a process in which the common attributes of more than one entities form a new entity. This newly formed entity is called generalized entity. Let's say we have two entities **Student** and **Teacher**. Attributes of Entity Student are: Name, Address & Grade, Attributes of Entity Teacher are: Name, Address & Salary



Specialization

Specialization is opposite to Generalization. It is a top-down approach in which one higher level entity can be broken down into two lower level entities. Specialization is used to identify the subset of an entity set that shares some distinguishing characteristics. The idea behind Specialization is to find the subsets of entities that have few distinguish attributes. For example – Consider an entity employee which can be further classified as sub-entities Technician, Engineer & Accountant because these sub entities have some distinguish attributes.



Aggregation

Aggregation is a process when relation between two entities is treated as a **single entity**. In the diagram above, the relationship between **Center** and **Course** together, is acting as an Entity, which is in relationship with another entity **Visitor**. Now in real world, if a Visitor or a Student visits a Coaching Center, he/she will never enquire about the center only or just about the course, rather he/she will ask enquire about both.

